1-on-1 Discussion Guide

A walkthrough plan for explaining your project to the instructor, organized module by module. Focus on showing you **understand** the crypto concepts, not just the code.

Suggested Flow (15–20 min)

```
1. Big picture (2 min)      -> What problem are we solving?
2. utils.py (1 min)         -> Foundation: SHA-256 and secure randomness
3. lamport.py (3 min)       -> The simplest hash-based signature
4. wots.py (4 min)          -> The key improvement over Lamport
5. merkle.py (4 min)        -> Lifting the one-time restriction
6. Security demo (3 min)    -> Why one-time matters
7. Results (3 min)          -> What we measured and observed
```

---

1. Big Picture (open with this)

> "Our goal was to build the fundamental building blocks behind post-quantum signature schemes like SPHINCS+. We implemented three schemes — Lamport, WOTS, and Merkle — entirely from scratch using SHA-256, then benchmarked and analyzed them."

Key point to land: These schemes are secure because they only rely on hash function one-wayness — unlike RSA/ECDSA which would be broken by Shor's algorithm on a quantum computer.

---

2. `utils.py` — The Foundation

Walk through briefly. Highlight:

- **`get_hash`**: "Everything depends on SHA-256 being one-way — given `H(x)`, you can't find `x`."
- **`hash_n`**: "This is used by WOTS for hash chains — applying SHA-256 iteratively `n` times."
- **`generate_random_bytes`**: "We use `secrets` instead of `random` because key generation requires cryptographic-quality randomness."

---

3. `lamport.py` — Lamport OTS

What to show

Open the file and walk through `generate_keypair` -> `sign` -> `verify`.

How to explain it

> "The secret key is 256 pairs of random blocks — one pair per bit of the SHA-256 hash. The public key is the hash of each block. To sign, we hash the message, then for each bit, we reveal the corresponding secret block. The verifier re-hashes each revealed block and checks it matches the public key."

Key points to emphasize

Topic	What to say
Why it's secure	"An attacker would need to find a preimage of a SHA-256 hash to forge a block, which is computationally infeasible."
Why signing is fast	"Signing doesn't compute any hashes — it just selects and copies pre-existing blocks. That's why it's 0.03 ms."
Sizes	"Signature = 256 x 32 = 8,192 bytes. Public key = 512 x 32 = 16,384 bytes. This is large — which is why WOTS was invented."
Limitation	"It's strictly one-time. Reusing the key leaks additional blocks. We demonstrate this in the security analysis."

Likely instructor question

> **Q: "Why is it one-time?"**
>
> A: "Each signature reveals 256 out of 512 secret blocks. A second signature reveals another 256, potentially different ones. After enough reuses, the attacker knows all 512 blocks and can forge any message. We verified this experimentally — after 10 reuses, forgery succeeds 78% of the time."

---

4. `wots.py` — Winternitz OTS

What to show

Open the file. Focus on `_get_message_digits`, `_get_checksum_digits`, and `sign`.

How to explain it

> "Instead of signing one bit at a time like Lamport, WOTS groups the hash into base-w digits. For w=16, each digit is 4 bits, so we only need 64 chains instead of 256 pairs. Each chain is a sequence of iterated hashes: the secret is the start, the public key is the end after w-1 hashes, and the signature is an intermediate point."

The checksum — the hardest part to explain

> "The checksum is crucial. Without it, an attacker who sees a signature with digit d=5 could forge a new message by replacing it with d=3 — because going from H^5(SK) to H^3(SK) doesn't require the secret key (you can just hash backward... wait, you can't hash backward, but you CAN just use fewer hashes on the same starting point). Actually, the attack works the OTHER direction: the attacker can INCREASE the digit from 5 to 7 by applying 2 more hashes to the signature. But the checksum = sum(w-1-d_i) would decrease if any d_i increases, and decreasing the checksum means you'd need to compute FEWER hashes on the checksum chains — which means going BACKWARD, which is impossible."
>
> [!TIP] **Simpler way to say it:** "The checksum creates a coupling: increasing any message digit forces a decrease in the checksum digit, which would require reversing a hash — which is impossible by assumption."

Key points to emphasize

Topic	What to say
Size reduction	"With w=16: 67 chains x 32 bytes = 2,144 B — that's 3.8x smaller than Lamport."
Time trade-off	"But each chain is 15 hashes long, so keygen/sign/verify are ~10x slower."
Parameter l1, l2	"l1 = ceil(256/log2(w)) covers message digits. l2 covers the checksum — typically 3 extra chains."

Likely instructor question

> **Q: "Why not just use w=256 for minimum size?"**
>
> A: "Because each chain would be 255 hashes long. KeyGen would need 67 x 255 ~ 17,000 hashes vs. 67 x 15 ~ 1,000 for w=16. Our benchmarks show w=16 is the sweet spot — it's what XMSS and SPHINCS+ use."

---

5. `merkle.py` — Merkle Signature Scheme

What to show

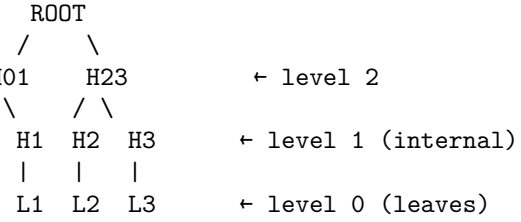
Open the file. Walk through `generate_keypair` (tree building), `sign` (auth path), and `verify` (root reconstruction).

How to explain it

> "The Merkle tree lets us turn many one-time keys into one reusable key. We generate 2^h OTS keypairs, hash each public key into a leaf, and build a binary hash tree. The root is the MSS public key. To sign the i-th message, we sign with OTS key i and include the authentication path — the h sibling hashes needed to reconstruct the root."

The authentication path — draw it!

> [!IMPORTANT] If possible, draw a small tree (h=3) on paper/whiteboard during the discussion. This makes the auth path immediately obvious.



> "If I sign with leaf 1 (L1), the auth path is [L0, H23] — the sibling at each level. The verifier computes H(L0 || L1) = H01, then H(H01 || H23) = ROOT, and checks it matches."

Key points to emphasize

Topic	What to say
Stateful tracking	"We have a `next_leaf` counter that ensures no leaf is ever reused. Once all 2^h leaves are used, signing raises an error."

Topic	What to say
KeyGen cost	"KeyGen is O(2^h) — we must generate all OTS keypairs upfront. This is the bottleneck."
Sign/Verify cost	"Sign and verify only do ONE OTS operation plus h hashes — effectively constant regardless of tree size."
Signature overhead	"Each tree level adds only 32 bytes (one SHA-256 hash) to the signature."

Likely instructor questions

Q: "Why is this called stateful?"

A: "Because the signer must track which leaf was used last. If the state is lost (e.g., after a crash), and a leaf is accidentally reused, the OTS key-reuse vulnerability applies. This is a real-world concern — it's why NIST also standardized SPHINCS+, which is stateless."

Q: "How does verification work without the full tree?"

A: "The verifier only needs the root (public key), the OTS public key, the OTS signature, and the auth path. They reconstruct the root bottom-up using h hashes and compare. They never need the full tree."

---

6. Security Demo

What to show

Run `PYTHONPATH=. python3 src/security_simulation.py` live.

How to explain it

"We sign 20 messages with the same Lamport key. Each signature leaks 256 of the 512 secret blocks. After 20 signings, we've observed enough blocks to forge a valid signature on any arbitrary message. The forged signature passes the legitimate verifier — demonstrating that key reuse completely breaks the scheme."

Connect to the quantitative analysis

"In our benchmarks, we ran 200 trials per value of k. The empirical forgery rate matches the theoretical formula $(1 - 2^{\{-k\}})\{256\}$ almost perfectly. By k=10, forgery succeeds ~78% of the time. By k=15, it's essentially certain."

---

7. Results to Highlight

Talking points for each key result

Result	What to say
WOTS w=16 vs Lamport	"WOTS(16) gives us 3.8x smaller signatures with sign+verify still under 0.6 ms. This is why real standards use w=16."
MSS KeyGen scaling	"KeyGen doubles with each tree level — it's O(2^h). But sign and verify are flat."

Result	What to say
Auth path overhead	"Each extra level costs only 32 bytes per signature. Going from h=4 to h=8 (16->256 messages) adds just 128 bytes."
Key-reuse attack	"Our simulation confirms the theory: after 10 reuses, 78% forgery rate; after 20, 100% with full key recovery."
Post-quantum security	"All our schemes rely only on SHA-256 being one-way. Grover's algorithm gives at most a quadratic speedup, so SHA-256 still provides ~128 bits of quantum security."

Common Tough Questions & Answers

"What is the security proof?"

> "Hash-based signatures have a clean reduction: if an attacker can forge a signature, they can invert SHA-256. Since SHA-256 inversion is believed to be hard (and Grover only gives sqrt speedup), the scheme is secure. We don't need any number-theoretic assumptions like factoring."

"How does this relate to SPHINCS+?"

> "SPHINCS+ uses exactly these building blocks — WOTS and Merkle trees — but adds a hypertree structure (a tree of trees) and a few-time signature layer (FORS) to make the scheme completely stateless. Our implementation is the educational core."

"Why not use a hash other than SHA-256?"

> "Any secure hash function works. SHA-256 provides 128-bit quantum security (Grover halves the bits). If we wanted more margin, we could use SHA-512, but 128-bit quantum security meets NIST Category 1."

"What are the practical limitations?"

> "Signature sizes are larger than RSA/ECDSA (KB vs hundreds of bytes). KeyGen for large trees is slow. The scheme is stateful — losing track of the leaf counter can lead to key reuse. SPHINCS+ addresses the statefulness problem at the cost of even larger signatures (~17 KB)."